CS5100 Final Project Phase 2 Report

Github Repository to Final Project : https://github.com/Fall25-CS5100-JesseAStern/final-project-fall-2025-super-awesome-team

1. Use Full Dataset

This stage involves comparing F1 score of the Random Forest Classifier in Phase 1 using two datasets `student-mat.csv` and `student-mat-mini.csv` . The result was a **40% boost** in F1 score when training the classifier with `student-mat.csv` , which is the larger dataset.

In [1]:
```
from sklearn.linear_model import LogisticRegression

# import libraries
import student_project as sp
from sklearn.metrics import f1_score
from sklearn.model_selection import train_test_split

from student_project import load_data
```

In [2]:
```
%%capture

# ready the Random Forest Classifier with large dataset
df_lg = sp.load_data("../datasets/student-mat.csv")
df_lg_clean = sp.preprocess_data(df_lg)
X_lg = df_lg_clean.drop(columns=["at_risk"]).values
y_lg = df_lg_clean["at_risk"].values
Xtr_l, Xte_l, ytr_l, yte_l = train_test_split(X_lg, y_lg,
test_size=0.3,
                                     random_state=42,
stratify=y_lg)
rf_lg = sp.RandomForest(n_estimators=5, max_depth=4,
                        sample_size=min(64, len(Xtr_l)),
random_state=42)
rf_lg.fit(Xtr_l, ytr_l)

# prepare RF Classifier with small dataset
df_sm = sp.load_data("../datasets/student-mat-mini.csv")
df_sm_clean = sp.preprocess_data(df_sm)
X_sm = df_sm_clean.drop(columns=["at_risk"]).values
y_sm = df_sm_clean["at_risk"].values
Xtr_s, Xte_s, ytr_s, yte_s = train_test_split(X_sm, y_sm,
test_size=0.3,
                                               random_state=42,
stratify=y_sm)
rf_sm = sp.RandomForest(n_estimators=5, max_depth=4,
                        sample_size=min(64, len(Xtr_s)),
random_state=42)
rf_sm.fit(Xtr_s, ytr_s)
```

In [3]:
```
import time

# evaluate speed and accuracy of RFC with large dataset
predict_start_lg = time.perf_counter()
yp_lg = rf_lg.predict(Xte_l)
f1_rf_lg = f1_score(yte_l, yp_lg, zero_division=0)
predict_end_lg = time.perf_counter()
predict_elapsed_lg = predict_end_lg - predict_start_lg

# evaluate speed and accuracy of RFC with small dataset
predict_start_sm = time.perf_counter()
yp_sm = rf_sm.predict(Xte_s)
f1_rf_sm = f1_score(yte_s, yp_sm, zero_division=0)
predict_end_sm = time.perf_counter()
predict_elapsed_sm = predict_end_sm - predict_start_sm

print("Random Forest Classifier with student-mat.csv")
print(f"F1 score: {f1_rf_lg:.2f}\t Elapsed time:
{predict_elapsed_lg:.2f} sec.")
print("\nRandom Forest Classifier with student-mat-mini.csv")
print(f"F1 score: {f1_rf_sm:.2f}\t Elapsed time:
{predict_elapsed_sm:.2f} sec.")

print(f"\nResult: F1 score BOOSTED {(f1_rf_lg / f1_rf_sm - 1) *
100:.2f}%")
```

```
Random Forest Classifier with student-mat.csv
F1 score: 0.40   Elapsed time: 30.66 sec.

Random Forest Classifier with student-mat-mini.csv
F1 score: 0.29   Elapsed time: 1.45 sec.

Result: F1 score BOOSTED 40.00%
```

2. Reduced Dataset Dimensionality with Feature Selection

This stage involves reducing the dimensionality of the dataset using feature importance approach. When calculating Gini impurity, we globally track the value for each node in the decision tree, providing the information of how each criterion is of more importance in terms of selecting the best feature to split. Then, we pick k features with the highest importance to build the random forest classifier.

As a result, the best k was 30, and the dataset with reduced dimensionality suffered 26.14% F1 score decrease but the elapsed time decreased 55.23%. This approach would be appropriate if runtime is prioritized than accuracy of the model.

In [4]:
```
%%capture

# prepare the test environment
df = load_data("../datasets/student-mat.csv")
df_processed = sp.preprocess_data(df)
X = df_processed.drop('at_risk', axis=1)
y = df_processed['at_risk']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
rf = sp.RandomForest(n_estimators=5, random_state=42,
    track_feature_importance=True
)
rf.fit(X_train, y_train)
```

In [5]:
```
# Get baseline performance
predict_start = time.perf_counter()
y_pred_full = rf.predict(X_test)
predict_end = time.perf_counter()
predict_elapsed_full = predict_end - predict_start
f1_full = f1_score(y_test, y_pred_full)
print(f"\nBaseline F1 Score: {f1_full:.2f}\t Elapsed time:
{predict_elapsed_full:.2f} sec.")
```

```
Baseline F1 Score: 0.38  Elapsed time: 50.69 sec.
```

In [6]:
```
# Get top features with high importance
importances = rf.get_feature_importances()
sorted_features = sorted(importances.items(), key=lambda x: x[1],
reverse=True)
top_features = sorted_features[:10]
print("Top 10 most important features:")
for i, (feature, importance) in enumerate(sorted_features[:10], 1):
    print(f"{i:2d}. {feature:30s}: {importance:.4f}")
```

```
Top 10 most important features:
 1. sex_F                          : 0.0949
 2. Mjob_at_home                   : 0.0709
 3. school_GP                      : 0.0633
 4. address_R                      : 0.0551
 5. reason_course                  : 0.0504
 6. famsize_GT3                    : 0.0459
 7. paid                           : 0.0455
 8. famsup                         : 0.0420
 9. Fedu                           : 0.0413
10. age                            : 0.0390
```

In [7]:
```
%%capture

# comparing different k values
k = 30  # about half the size of baseline dataset
top_features = rf.select_top_k_features(k)
X_train_reduced = X_train[top_features]
X_test_reduced = X_test[top_features]

# train new Random Forest classifier with reduced dataset
max_tree_depth = min(k-1, 10)
rf_reduced = sp.RandomForest(
    n_estimators=5,
    max_depth=max_tree_depth,
    random_state=42
)
rf_reduced.fit(X_train_reduced, y_train)
```

In [8]:
```
# Get Performance
predict_start = time.perf_counter()
y_pred_reduced = rf_reduced.predict(X_test_reduced)
predict_end = time.perf_counter()
predict_elapsed_reduced = predict_end - predict_start
f1_reduce = f1_score(y_test, y_pred_reduced)
print(f"\nReduced F1 Score: {f1_reduce:.2f}\t Elapsed time:
{predict_elapsed_reduced:.2f} sec.")
```

```
Reduced F1 Score: 0.30   Elapsed time: 28.57 sec.
```

In [9]:
```
# Compare with baseline and reduced dataset

print("SUMMARY")
print(f"{'Model':<10} {'# features':<12} {'F1 Score':<12} {'Elapsed
time (sec)':<12}")
print(f"{'Baseline':<10} {X_train.shape[1]:<12} {f1_full:<12.4f}
{predict_elapsed_full:<12.2f}")
print(f"{'Reduced':<10} {X_train_reduced.shape[1]:<12} {f1_reduce:
<12.4f} {predict_elapsed_reduced:<12.2f} ")

print(f"\nResult: Dataset with reduced dimensionality suffered
{(f1_full / f1_reduce - 1) * 100:.2f}% F1 score decrease, but the
elapsed time also decreased {(predict_elapsed_reduced /
predict_elapsed_full) * 100:.2f}%.")
```

```
SUMMARY
Model      # features   F1 Score     Elapsed time (sec)
Baseline   48           0.3750       50.69
Reduced    30           0.2973       28.57

Result: Dataset with reduced dimensionality suffered 26.14% F1
score decrease, but the elapsed time also decreased 56.35%.
```

3. Stacking

(following the textbook as reference)

In this stage, the following base models will be trained.
* Logistic Regression (sklearn library)
* Support Vector Machines (sklearn library)
* Random Forest (implemented in phase 1)

Then, the predictions made by the three base models will be augmented into the dataset, with which the meta model (Random Forest in this case) will be trained on.

The result was a 34% boost in F1 score compared to baseline Random Forest classifier.

In [19]:
```
# prepare the test environment
import pandas as pd
from student_project import *
from sklearn.linear_model import LogisticRegression
from sklearn import svm

df = load_data("../datasets/student-mat.csv")
df_processed = preprocess_data(df)
X = df_processed.drop('at_risk', axis=1)
y = df_processed['at_risk']
X_train, X_test, y_train, y_test = train_test_split(X, y,
random_state=42)
```

```
File found in ../datasets/student-mat.csv,
loading...
```

In [20]:
```
# train Logistic Regression and generate predictions
lr = LogisticRegression(random_state=42)
lr.fit(X_train, y_train)
predictions_lr = lr.predict(X_test)
```

In [21]:
```
# train SVM
svc = svm.SVC(random_state=42)
svc.fit(X_train, y_train)
predictions_svc = svc.predict(X_test)
```

In [22]:
```
%%capture
# train random forest
rf = sp.RandomForest(random_state=42)
rf.fit(X_train, y_train)
predictions_rf = rf.predict(X_test)
```

In [14]:
```
print(f"Random Forest: {predictions_rf}")
```

```
Random Forest: [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 1 0 0 1 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0
0 0 0 0 0 0
 1 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0]
```

In [15]:
```
# add predictions to dataset
X_aug = X
lr_series = pd.Series(predictions_lr)
X_aug['lr'] = lr_series
svc_series = pd.Series(predictions_svc)
X_aug['svc'] = svc_series
rf_series = pd.Series(predictions_rf)
X_aug['rf'] = rf_series
X_train, X_test, y_train, y_test = train_test_split(X_aug, y,
random_state=42)
```

In [16]:
```
%%capture
# train random forest with new dataset
meta_rf = RandomForest(random_state=42)
meta_rf.fit(X_train, y_train)
predictions_meta_rf = meta_rf.predict(X_test)
```

In [17]:
```
baseline_rf_score = f1_score(y_test, predictions_rf)
meta_rf_score = f1_score(y_test, predictions_meta_rf)
print(f"Meta Random Forest F1 Score:     {meta_rf_score:.2f}")
print(f"Baseline Random Forest F1 Score: {baseline_rf_score:.2f}")
print(f"Result: F1 score was increased by {(baseline_rf_score /
meta_rf_score) * 100:.2f}% when stacking was applied.")
```

```
Meta Random Forest F1 Score:     0.50
Baseline Random Forest F1 Score: 0.17
Result: F1 score was increased by 34.04% when stacking was
applied.
```

Reflection

What was your favorite/least favorite part of the project?

My most AND least favorite part of the project was building the decision tree from scratch. Implementing a data structure is one of my most favorite approach in studying, and I learned a lot from implementing decision trees. The reason why it was the least favorite was because when I moved onto implementing Random Forest classifiers I constantly had to restructure the whole class to make the classifier work, which was very challenging and time-consuming.

On what topic from the course did your perspective change the most from before to after the project? Did your opinion of the usefulness of this topic go up or down after working with it within the context of the project?

Supervised learning was definitely had the most perspective change before and after the project, since it was the main topic of the final project. I had some experience in supervised learning when I was taking natural language processing, but I did not favor the approach due to excessive preprocessing it needs to accurately fit and predict. But for datasets similar to what we have dealt with the final project, I realized the advantages of supervised learning in terms of high accuracy and clear result.

If you had more time, which scope item (or item from phase 1) that you implemented would you wish to improve the implementation of further and how might you try to do this?

If I had more time, I would have focused more on pseudocoding the decision tree and random forest implementation before diving into python code, which is a common mistake I make when I am coding. I opened up a Jupyter notebook and implemented as I go, but now I realize this was the wrong turn to take.

If you had more time, what new scope item/ techniques would you like to try to apply and why?

I would have tried applying transformers. I skimmed through some readings describing what they were during Natural Language Processing course, but have not had the chance to actually implement one myself.

Any recommendations to improve the project experience for future students? (optional)

Code reviews could be one of the ways the class would benefit from the project experience. Even if the code works, I was often unsure if this was the right way to write it and was curious about ways to improve it. And I am not saying that we should only receive reviews; I believe if students could have a peer review between themselves it would be a great experience.

Roughly how many hours did you spend one phase 1 and phase 2 of the project respectively? This can be a 1 sentences answer and will not impact grade (optional)

* Phase 1: ~30 hours
* Phase 2: ~20 hours

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.